

React bilan noldan sayt qurish

HTML/CSS/JS bilgan dasturchi uchun amaliy qo'llanma

Kirish

Agar siz HTML, CSS va JavaScript'ni bilasiz, lekin real loyihada React'ni qanday ishlatishni tasavvur qila olmayotgan bo'lsangiz — bu qo'llanma aynan siz uchun.

Ko'pchilik React'ni o'rganishni rasmiy hujjatlardan yoki video darslardan boshlaydi, u yerda hammasi allaqachon tayyor holda, chiroyli qilib ko'rsatiladi: `npm create vite@latest`, bitta komponent, bitta useState — va tamom. Lekin real loyiha — masalan, haqiqiy biznes uchun sayt — bunchalik silliq bo'lmaydi. Sizda: bir nechta sahifa, forma, ma'lumotlar ro'yxati, rasm galereyasi, mobil moslashuv, va "buni qanday qismlarga bo'lish kerak" degan doimiy savol bo'ladi.

Bu qo'llanma nazariyani takrorlamaydi — React hujjatlarida bu allaqachon bor. Buning o'rniga, boshidan oxirigacha **bitta real loyihani** — kichik biznes uchun ko'p sahifali sayt (masalan, mehmonxona yoki xizmat ko'rsatuvchi kompaniya sayti) — birga quramiz. Har bir bobda nafaqat "nima qilish kerak", balki "nega aynan shunday" degan savolga javob beramiz, chunki aynan shu qaror qabul qilish qobiliyati boshlang'ich va o'rta darajadagi dasturchini ajratib turadi.

Bu qo'llanma sizga bermaydigan narsa: React'ning har bir API'sini qatma-qat tushuntirish, Redux/Zustand kabi murakkab state management kutubxonalari, server-side rendering (Next.js). Bular — keyingi qadam uchun mavzular.

Bu qo'llanma sizga beradigan narsa: bitta sayt, noldan production'gacha, real qarorlar bilan — struktura, komponentlarga bo'lish, forma bilan ishlash, deploy qilish.

Mundarija

1-qism: Poydevor - 1-bob: Nega React (va qachon kerak emas) - 2-bob: Loyihani sozlash (Vite, papka strukturasini) - 3-bob: Komponent tafakkuri — HTML'ni qismlarga qanday bo'lish kerak

2-qism: Qurilish - 4-bob: Birinchi komponentlar — Header, Footer, Layout - 5-bob: Props bilan ishlash — statik ma'lumotdan dinamikgacha - 6-bob: State va useState — interaktiv qismlar (menyu, tab, modal) - 7-bob: Ro'yxatlar va map() — xonalar/mahsulotlar katalogini chiqarish - 8-bob: Forma yaratish — buyurtma/bog'lanish formasi va validatsiya - 9-bob: useEffect va tashqi ma'lumot — statik JSON'dan API'gacha

3-qism: Jilolash va chiqarish - 10-bob: Routing — bir nechta sahifa (React Router) - 11-bob: Responsive va mobil moslashuv React komponentlarida - 12-bob: Xatolar bilan ishlash va bo'sh holatlar (loading, error, empty) - 13-bob: Deploy qilish — Vercel'ga chiqarish, domen ulash - 14-bob: Keyingi qadamlar — nimani o'rganish kerak

Ilova: Tez-tez uchraydigan xatolar va ularning yechimi

1-BOB: Nega React (va qachon kerak emas)

Muammo, React yechadigan

HTML/CSS/JS bilan sayt qurganingizda, vaqt o'tishi bilan bitta muammoga duch kelasiz: **bir xil HTML bo'lagi bir nechta joyda takrorlanadi**, va uni o'zgartirish kerak bo'lganda, hamma nusxalarni qo'lda topib, birma-bir tahrirlashga to'g'ri keladi.

Masalan, saytingizda "xona kartasi" bor — rasm, nom, narx, "Bron qilish" tugmasi. Agar sizda 10 ta xona bo'lsa, HTML'da bu blok 10 marta yozilgan bo'ladi. Narx formatini o'zgartirmoqchi bo'lsangiz — 10 joyni tahrirlaysiz. Yangi xona qo'shmoqchi bo'lsangiz — yana bir blokni qo'lda nusxalaysiz, kamida bitta atributni o'zgartirishni unutib, sayt sinadi.

React bu muammoni **komponent** tushunchasi bilan yechadi: siz "xona kartasi"ni bir marta, funksiya sifatida yozasiz, va unga ma'lumot (nom, narx, rasm) tashqaridan beriladi. 10 ta xona — 10 marta yozilgan HTML emas, balki bitta komponentga 10 marta chaqirilgan ma'lumot.

```
// HTML/JS'da – har bir xona uchun qo'lda yozilgan blok (soddalashtirilgan)
const room1 = `

Bu misolda hali katta farq sezilmasligi mumkin, lekin 5-bobdan boshlab, kartalar soni o'sganda va ma'lumot massiv (array) sifatida kelganda, farq keskin ko'rinadi.



## Qachon React kerak EMAS



Bu muhim — chunki ko'pchilik har bir loyihaga React ishlatishga o'rganib qoladi, garchi kerak bo'lmasa ham.



Agar sizning saytingiz: - 1-3 ta statik sahifadan iborat (masalan, oddiy "biz haqimizda" sayti) - Interaktivlik deyarli yo'q (forma yo'q, filtr yo'q, dinamik ro'yxat yo'q) - SEO juda muhim va sahifa tezligi hal qiluvchi (statik HTML har doim tezroq yuklanadi)



— bunday holatda oddiy HTML/CSS/JS yoki statik sayt generatori (masalan Astro) ko'proq mos keladi. React bunday loyihaga ortiqcha murakkablik va katta JavaScript fayl hajmini qo'shadi.



React aynan quyidagi holatlarda foydali: - Sayt ko'p sahifali va takrorlanuvchi bloklar ko'p (kartalar, ro'yxatlar) - Foydalanuvchi bilan interaktivlik bor: forma, filtr, tab, modal, savat - Ma'lumot tez-tez o'zgaradi (API'dan keladi, foydalanuvchi harakatiga qarab yangilanadi) - Kelajakda loyiha kengayishi kutilmoqda (yangi sahifalar, yangi funksiyalar)



Amaliy qoida: agar loyihangizda "bu blokni yana bir joyda takrorlash kerakmi" degan savol uchinchi marta tug'ilsa — React vaqti keldi.



## Xulosa



React — sehrli vosita emas, balki takrorlanuvchi va o'zgaruvchan interfeyslarni boshqarish uchun vosita. Keyingi bobda loyihani noldan sozlaymiz va nega aynan Vite tanlanganini ko'ramiz.


```

2-BOB: Loyihani sozlash

Vite nima va nega Create React App emas

Bir necha yil oldin React loyihalarini `create-react-app` bilan boshlash odat edi. Hozir bu vosita rasman eskirgan (deprecated) hisoblanadi — sekin ishlaydi va yangilanishlar to'xtatilgan.

Uning o'rnini **Vite** egallagan. Vite loyihani soniyalar ichida ishga tushiradi (create-react-app esa ko'pincha 30-60 soniya kutishni talab qilardi) va development jarayonida o'zgarishlarni deyarli bir zumda ko'rsatadi (Hot Module Replacement).

Loyihani yaratish

Terminal orqali:

```
npm create vite@latest mening-saytim -- --template react-ts
cd mening-saytim
npm install
npm run dev
```

`react-ts` shablonini tanlashning sababi — TypeScript. Agar siz hali TypeScript bilmasangiz, xavotir olmang: bu qo'llanmada TypeScript minimal darajada, faqat eng foydali joylarda ishlatiladi (masalan, props tiplari). Sof JavaScript bilan ham davom etishingiz mumkin — buning uchun `--template react` shablonini tanlang, qo'llanmadagi kodlarda TypeScript belgilarini (`: string`, `: number` kabi) shunchaki e'tiborsiz qoldiring.

Papka strukturasi — nega tartib muhim

Vite loyihani yaratganda, sizga quyidagi asosiy papkalarni beradi:

```
mening-saytim/
├── public/           ← o'zgarmaydigan fayllar (favicon, statik rasm)
├── src/
│   ├── assets/      ← komponentlar ishlatadigan rasm/ikonkalar
│   ├── App.tsx      ← asosiy komponent
│   └── main.tsx     ← ilovaning kirish nuqtasi
├── index.html
└── package.json
```

Loyiha o'sishi bilan, `src/` ichida quyidagi strukturani qo'shib borish tavsiya etiladi:

```
src/
├── components/      ← qayta ishlatiladigan kichik bo'laklar (Button, Card, Header)
├── pages/           ← har bir sahifaga mos katta komponentlar (Home, About, Contact)
├── data/            ← statik ma'lumotlar (JSON yoki JS massivlar)
├── hooks/           ← maxsus React hook'lar (agar kerak bo'lsa)
└── styles/          ← CSS fayllar
```

Nega bu muhim: loyiha 5-10 komponentdan oshganda, hamma faylni `src/` ichiga tekis joylashtirish (flat structure) faylni topishni qiyinlashtiradi. Boshidanoq `components/` va `pages/` ajratib qo'yish, 6 oydan keyin loyihaga qaytganingizda ham tez orientatsiya olishga yordam beradi.

Birinchi tekshiruv

`npm run dev` buyrug'idan keyin brauzerda `localhost:5173` manzilini oching. Vite'ning standart sahifasi ko'rinsa — muhit tayyor.

Keyingi bobda birinchi real komponentni yozamiz va "komponent tafakkuri" nima ekanini ko'ramiz — bu React'dagi eng muhim ko'nikma, sintaksisdan ham muhimroq.

3-BOB: Komponent tafakkuri — HTML'ni qismlarga qanday bo'lish kerak

Eng ko'p uchraydigan xato

React o'rganuvchilarning aksariyati sintaksisni (JSX, props, state) tezda o'zlashtiradi, lekin ular haqiqiy qiynaladigan joy — **butun sahifani qanday komponentlarga bo'lish kerak**, degan savol. Bu ko'nikma hech qanday video darsda to'liq o'rgatilmaydi, chunki u amaliyot bilan keladi.

Mashq: bitta sahifani tahlil qilish

Tasavvur qiling, sizda quyidagi tuzilishga ega mehmonxona saytining bosh sahifasi bor:

```
[ Logo – Menyu – Bog'lanish tugmasi ]      ← header
[ Katta rasm + sarlavha + "Bron qilish" ]    ← hero
[ Xona 1 ] [ Xona 2 ] [ Xona 3 ]            ← xonalar ro'yxati
[ Izoh 1 ] [ Izoh 2 ]                       ← mijozlar izohi
[ Manzil – Telefon – Ijtimoiy tarmoqlar ]   ← footer
```

Buni komponentlarga bo'lishning ikkita noto'g'ri yo'li bor:

- 1-xato: hammasini bitta komponentga yozish.** `App.tsx` 400 qatorlik JSX bilan to'lib ketadi. Birona qismni o'zgartirish uchun butun faylni qidirishga to'g'ri keladi.
- 2-xato: haddan tashqari mayda bo'lish.** Har bir `<p>`, har bir ikonka uchun alohida komponent yaratish. Bu ortiqcha fayllar va murakkablik keltirib chiqaradi, hech qanday foyda bermaydi.

To'g'ri yondashuv: ikkita savol

Har bir blok uchun o'zingizga ikkita savol bering:

- 1. Bu blok qayta ishlatiladimi?** Agar "Xona kartasi" 3 marta takrorlansa — bu aniq komponent kandidati. Agar "Header" sahifada faqat bir marta bo'lsa ham, u alohida komponent bo'lishi kerak — chunki u mantiqiy jihatdan mustaqil bo'lak.
- 2. Bu blok o'zining ma'lumoti yoki state'iga egami?** Agar "Menyu" ochiq/yopiq holatga ega bo'lsa (mobil burger menyu), bu holat faqat o'sha komponentga tegishli — demak, u alohida komponent bo'lishi kerak, shunda holat boshqa qismlarga aralashmaydi.

Yuqoridagi sahifa uchun to'g'ri bo'linish:

```
App
├── Header
│   └── MobileMenu (agar burger menyu bo'lsa)
├── Hero
├── RoomsSection
│   └── RoomCard (3 marta chaqiriladi)
├── TestimonialsSection
│   └── TestimonialCard (2 marta chaqiriladi)
└── Footer
```

Bu struktura — **komponent daraxti** deb ataladi. `App` — ildiz, qolganlari uning "farzandlari".

Fayllarga taqsimlash

Har bir komponent — o'z faylida:

```
src/
├── components/
│   ├── Header.tsx
│   ├── Hero.tsx
│   ├── RoomCard.tsx
│   ├── TestimonialCard.tsx
│   └── Footer.tsx
├── sections/
│   ├── RoomsSection.tsx
│   └── TestimonialsSection.tsx
└── App.tsx
```

Nega "kartalar" alohida `RoomCard.tsx` , alohida `TestimonialCard.tsx` fayl: kelajakda dizaynchi yoki siz o'zingiz faqat "xona kartasi" ustida ishlashingiz kerak bo'lganda, faylni nomidan darhol topasiz — butun `App.tsx` ichida qidirmaysiz.

Amaliy topshiriq

Keyingi bobga o'tishdan oldin: o'zingiz tanlagan biror real sayt (masalan sevimli internet-do'kon yoki xizmat sayti)ni oching va uni qog'ozga komponent daraxti sifatida chizib ko'ring. Bu mashqni bajarmasdan keyingi bobga o'tmang — bu qo'llanmaning eng muhim ko'nikmasi shu yerda shakllanadi.

4-BOB: Birinchi komponentlar — Header, Footer, Layout

Layout komponenti — nega kerak

Ko'p sahifali saytda Header va Footer deyarli har bir sahifada takrorlanadi. Buni har bir sahifa komponentida qayta yozish o'rniga, **Layout** komponenti yaratamiz — u Header va Footer'ni o'z ichiga oladi, sahifa mazmuni esa uning "ichida" (children) beriladi.

```
// components/Layout.tsx
import { ReactNode } from "react";
import Header from "../Header";
import Footer from "../Footer";

type LayoutProps = {
  children: ReactNode;
};

function Layout({ children }: LayoutProps) {
  return (
    <>
      <Header />
      <main>{children}</main>
      <Footer />
    </>
  );
}

export default Layout;
```

children — bu React'ning maxsus prop'i, komponent ichiga joylashtirilgan har qanday JSX'ni bildiradi. Bu tushuncha keyingi boblarda yana uchraydi, shuning uchun uni yaxshi tushunib olish muhim.

Header komponenti

```
// components/Header.tsx
function Header() {
  return (
    <header className="site-header">
      <div className="logo">MehmonUy</div>
      <nav>
        <a href="/">Bosh sahifa</a>
        <a href="/xonalar">Xonalar</a>
        <a href="/boglanish">Bog'lanish</a>
      </nav>
    </header>
  );
}

export default Header;
```

Diqqat qiling: hozircha havolalar oddiy `<a>` teglari bilan yozilgan. 10-bobda React Router kiritilganda, bularni `<Link>` komponentiga almashtiramiz — sababi shunda: oddiy `<a>` bosilganda, brauzer BUTUN sahifani qayta yuklaydi (sekin), `<Link>` esa faqat kerakli qismni yangilaydi.

Footer komponenti

```
// components/Footer.tsx
function Footer() {
  return (
    <footer className="site-footer">
      <p>&copy; 2026 MehmonUy. Barcha huquqlar himoyalangan.</p>
      <div className="socials">
        <a href="https://instagram.com">Instagram</a>
        <a href="https://t.me">Telegram</a>
      </div>
    </footer>
  );
}

export default Footer;
```

Bularni birlashtirish

```
// App.tsx
import Layout from "../components/Layout";

function App() {
  return (
    <Layout>
      <h1>Xush kelibsiz</h1>
      <p>Bu yerda sahifa mazmuni bo'ladi.</p>
    </Layout>
  );
}

export default App;
```

Shu bosqichda saytda Header, mazmun va Footer ko'rinishi kerak. Keyingi bobda **props** orqali komponentlarga tashqaridan ma'lumot berishni chuqurroq ko'ramiz — buning yordamida statik matnlarni dinamik ma'lumotga aylantiramiz.

5-BOB: Props bilan ishlash — statik ma'lumotdan dinamikgacha

Props nima

Props (properties) — komponentga tashqaridan beriladigan "argumentlar". Xuddi funksiyaga parametr uzatgandek.

```
type ButtonProps = {
  text: string;
  onClick: () => void;
};

function Button({ text, onClick }: ButtonProps) {
  return <button onClick={onClick}>{text}</button>;
}

// Ishlatish:
<Button text="Bron qilish" onClick={() => alert("Bosildi!")} />
```

Real misol: Hero komponentini dinamik qilish

3-bobda yaratgan Hero komponenti statik matn bilan yozilgan edi. Buni props orqali qayta ishlatiladigan qilamiz:

```
// Oldin — statik, faqat bosh sahifada ishlaydi
function Hero() {
  return (
    <section className="hero">
      <h1>MehmonUy'ga xush kelibsiz</h1>
      <p>Tabiat qo'ynida dam oling</p>
    </section>
  );
}

// Keyin — props orqali, istalgan sahifada ishlatiladi
type HeroProps = {
  title: string;
  subtitle: string;
};

function Hero({ title, subtitle }: HeroProps) {
  return (
    <section className="hero">
      <h1>{title}</h1>
      <p>{subtitle}</p>
    </section>
  );
}

// Bosh sahifada:
<Hero title="MehmonUy'ga xush kelibsiz" subtitle="Tabiat qo'ynida dam oling" />

// Xonalar sahifasida, xuddi shu komponent:
<Hero title="Bizning xonalar" subtitle="Har bir didga mos variant" />
```

Nega bu muhim: endi bitta Hero komponenti butun saytda ishlatiladi, faqat matn o'zgaradi. Dizaynni o'zgartirmoqchi bo'lsangiz (masalan fon rasmini), buni FAQAT bitta joyda — `Hero.tsx` faylida qilasiz, va o'zgarish barcha sahifalarda ko'rinadi.

Ixtiyoriy props va standart qiymatlar

Ba'zan prop majburiy emas. Masalan, rasm bo'lmasa ham Hero ishlashi kerak:

```
type HeroProps = {
  title: string;
  subtitle: string;
  backgroundImage?: string; // ? belgisi — ixtiyoriy
};

function Hero({ title, subtitle, backgroundImage = "/default-bg.jpg" }: HeroProps) {
  return (
    <section className="hero" style={{ backgroundImage: `url(${backgroundImage})` }}>
```



```
    <h1>{title}</h1>
    <p>{subtitle}</p>
  </section>
);
}
```

Keng tarqalgan xato: props'ni ortiqcha uzatish (prop drilling)

Agar `App` → `Layout` → `Page` → `Section` → `Card` kabi chuqur zanjirda bitta prop'ni har bir qatlamdan qo'lda o'tkazishga to'g'ri kelsa — bu "prop drilling" deb ataladi va kod yozishni og'irlashtiradi. Kichik va o'rta loyihalarda bu muammo kam uchraydi (odatda 2-3 qatlamdan oshmaydi), lekin agar sezsangiz — bu React Context API'ni o'rganish vaqti kelgani degani (14-bobda eslatib o'tiladi).

Keyingi bobda — **state** tushunchasi, ya'ni komponent o'zining ichki "xotirasi"ga ega bo'lishi, foydalanuvchi harakatiga javob berishi qanday ishlashini ko'ramiz.

6-BOB: State va useState — interaktiv qismlar

Props va state orasidagi farq

Bu ikkita tushunchani chalkashtirish — boshlang'ichlarning eng ko'p uchraydigan xatosi:

- **Props** — komponentga TASHQARIDAN beriladi, komponentning o'zi uni o'zgartira olmaydi (faqat o'qiy oladi).
- **State** — komponentning O'ZIGA tegishli, o'zgarishi mumkin bo'lgan ma'lumot. Masalan: mobil menyu ochiqmi-yopiqmi, forma maydoniga nima kiritilgan, qaysi tab tanlangan.

Birinchi misol: mobil burger menyu

```
import { useState } from "react";

function Header() {
  const [isMenuOpen, setIsMenuOpen] = useState(false);

  return (
    <header className="site-header">
      <div className="logo">MehmonUy</div>

      <button
        className="burger-button"
        onClick={() => setIsMenuOpen(!isMenuOpen)}
      >
        {isMenuOpen ? "x" : "≡"}
      </button>

      <nav className={isMenuOpen ? "nav-open" : "nav-closed"}>
        <a href="/">Bosh sahifa</a>
        <a href="/xonalar">Xonalar</a>
        <a href="/boglanish">Bog'lanish</a>
      </nav>
    </header>
  );
}
```

`useState(false)` — boshlang'ich qiymati `false` bo'lgan state yaratadi. `isMenuOpen` — hozirgi qiymatni o'qish uchun, `setIsMenuOpen` — uni o'zgartirish uchun ishlatiladi.

Muhim qoida: state'ni hech qachon to'g'ridan-to'g'ri o'zgartirmang (`isMenuOpen = true` — NOTO'G'RI). Har doim `set` funksiyasi orqali o'zgartiring. To'g'ridan-to'g'ri o'zgartirish React'ga "qayta chizish kerak" signalini bermaydi, va interfeys yangilanmaydi.

Ikkinchi misol: tab'lar (masalan "Xonalar / Uy-joylar / Bog'lar" tanlovi)

```
function Tabs() {
  const [activeTab, setActiveTab] = useState("xonalar");

  return (
    <div>
      <div className="tab-buttons">
        <button onClick={() => setActiveTab("xonalar")}>Xonalar</button>
        <button onClick={() => setActiveTab("uyjoylar")}>Uy-joylar</button>
        <button onClick={() => setActiveTab("boglar")}>Bog'lar</button>
      </div>

      <div className="tab-content">
        {activeTab === "xonalar" && <p>Xonalar ro'yxati bu yerda</p>}
        {activeTab === "uyjoylar" && <p>Uy-joylar ro'yxati bu yerda</p>}
        {activeTab === "boglar" && <p>Bog'lar ro'yxati bu yerda</p>}
      </div>
    </div>
  );
}
```

Nega state komponent ichida "yashaydi"

Har bir komponentning o'z state'i mustaqil. Agar sahifada 3 ta `RoomCard` bo'lsa va har birida "Batafsil" tugmasi bosilganda tavsif ochilsa (accordion), har bir karta o'zining `isOpen` state'iga ega bo'ladi — bitta kartani ochish boshqalariga ta'sir qilmaydi. Bu

React'ning kuchli tomonlaridan biri: har bir komponent nusxasi o'z holatini mustaqil boshqaradi.

Keyingi bobda — ro'yxatlarni qanday chiqarish, ya'ni 10 ta xonani qo'lda 10 marta yozish o'rniga, massiv orqali avtomatik chiqarish.

7-BOB: Ro'yxatlar va map() — katalogni chiqarish

Muammo

1-bobda aytilganidek, real muammo — bir xil komponentni ko'p marta, turli ma'lumot bilan chiqarish. Buning yechimi — JavaScript'ning `map()` metodi.

Ma'lumotni alohida faylga chiqarish

Avval, ma'lumotni komponentdan ajratib, alohida faylga joylashtiramiz — bu keyinchalik uni API'dan olishga o'tishni osonlashtiradi (9-bobda ko'ramiz):

```
// data/rooms.ts
export type Room = {
  id: number;
  name: string;
  price: number;
  capacity: number;
  image: string;
};

export const rooms: Room[] = [
  { id: 1, name: "Standart xona", price: 350000, capacity: 2, image: "/rooms/standart.jpg" },
  { id: 2, name: "Lyuks xona", price: 600000, capacity: 4, image: "/rooms/lyuks.jpg" },
  { id: 3, name: "Oilaviy xona", price: 850000, capacity: 6, image: "/rooms/oilaviy.jpg" },
];
```

RoomCard komponenti

```
// components/RoomCard.tsx
import { Room } from "../data/rooms";

function RoomCard({ name, price, capacity, image }: Room) {
  return (
    <div className="room-card">
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p>{capacity} kishigacha</p>
      <p className="price">{price.toLocaleString()} so'm / kecha</p>
      <button>Bron qilish</button>
    </div>
  );
}

export default RoomCard;
```

Ro'yxatni chiqarish

```
// sections/RoomsSection.tsx
import { rooms } from "../data/rooms";
import RoomCard from "../components/RoomCard";

function RoomsSection() {
  return (
    <section className="rooms-section">
      <h2>Bizning xonalar</h2>
      <div className="rooms-grid">
        {rooms.map((room) => (
          <RoomCard key={room.id} {...room} />
        ))}
      </div>
    </section>
  );
}

export default RoomsSection;
```

key prop — nega majburiy va nega muhim

`key={room.id}` — bu React'ga har bir elementni "tanib olish" uchun kerak. Agar `key` bo'lmasa, React konsolda ogohlantirish beradi va ro'yxat o'zgarganda (masalan bitta xona o'chirilganda) noto'g'ri elementni yangilashi mumkin.

Keng tarqalgan xato: `key` sifatida massiv indeksini ishlatish (`key={index}`). Bu ro'yxat statik bo'lsa (hech qachon tartib o'zgarmasa) ishlaydi, lekin ro'yxat filtrlansa yoki tartiblansa, xatoliklarga olib keladi. Har doim ma'lumotning o'ziga xos ID'sidan (`room.id`) foydalaning.

`{...room}` nima qiladi

Bu — "spread" sintaksisi, `room` obyektining barcha maydonlarini (`name` , `price` , `capacity` , `image`) alohida props sifatida uzatadi. Bu quyidagiga teng:

```
<RoomCard name={room.name} price={room.price} capacity={room.capacity} image={room.image} />
```

Spread sintaksisi kodni qisqartiradi, lekin komponent qaysi props kutayotganini yashiradi — shuning uchun katta jamoada ishlaganda, ba'zilar buning o'rniga har bir prop'ni alohida yozishni afzal ko'radi. Ikkala yondashuv ham to'g'ri, tanlov — jamoa yoki shaxsiy uslubga bog'liq.

Keyingi bobda — foydalanuvchidan ma'lumot olish: bron qilish formasi va uni tekshirish (validatsiya).

8-BOB: Forma yaratish — buyurtma formasi va validatsiya

Controlled input — React'da forma qanday ishlaydi

Oddiy HTML'da forma maydonlari o'z holatini o'zi saqlaydi. React'da esa har bir maydon qiymati **state orqali boshqariladi** — bu "controlled input" deb ataladi.

```
import { useState } from "react";

function BookingForm() {
  const [name, setName] = useState("");
  const [phone, setPhone] = useState("");
  const [guestCount, setGuestCount] = useState(1);

  function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    console.log({ name, phone, guestCount });
    // Bu yerda ma'lumotni serverga yoki Telegram botga yuborish logikasi bo'ladi
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Ismingiz"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <input
        type="tel"
        placeholder="Telefon raqam"
        value={phone}
        onChange={(e) => setPhone(e.target.value)}
      />
      <input
        type="number"
        min={1}
        value={guestCount}
        onChange={(e) => setGuestCount(Number(e.target.value))}
      />
      <button type="submit">Yuborish</button>
    </form>
  );
}
```

`e.preventDefault()` — brauzerning standart xatti-harakatini (sahifani qayta yuklashni) to'xtatadi, chunki biz formani JavaScript orqali qayta ishlaymiz.

Validatsiya — foydalanuvchi xatosini oldini olish

Real loyihada foydalanuvchi bo'sh forma yuborishi, noto'g'ri telefon raqam kiritishi mumkin. Buni oldini olish uchun, yuborishdan oldin tekshiruv qo'shamiz:

```
const [errors, setErrors] = useState<{ name?: string; phone?: string }>({});

function validate() {
  const newErrors: { name?: string; phone?: string } = {};

  if (name.trim().length < 2) {
    newErrors.name = "Ism kamida 2 ta harfdan iborat bo'lishi kerak";
  }

  const phoneRegex = /^\\+?[0-9]{9,12}$/;
  if (!phoneRegex.test(phone.replace(/\\s/g, ""))) {
    newErrors.phone = "Telefon raqam noto'g'ri formatda";
  }

  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
}

function handleSubmit(e: React.FormEvent) {
```

```
e.preventDefault();
if (validate()) {
  console.log("Forma to'g'ri, yuborilmoqda:", { name, phone, guestCount });
}
}
```

Xatolarni JSX'da ko'rsatish:

```
<input
  type="text"
  placeholder="Ismingiz"
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
{errors.name && <span className="error">{errors.name}</span>}
```

Yuborish holatini boshqarish (loading state)

Forma serverga (yoki Telegram bot API'siga) ma'lumot yuborayotganda, foydalanuvchi tugmani qayta-qayta bosishining oldini olish kerak:

```
const [isSubmitting, setIsSubmitting] = useState(false);

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  if (!validate()) return;

  setIsSubmitting(true);
  try {
    // await fetch(...) – server yoki botga yuborish
    console.log("Yuborildi");
  } finally {
    setIsSubmitting(false);
  }
}

// JSX:
<button type="submit" disabled={isSubmitting}>
  {isSubmitting ? "Yuborilmoqda..." : "Yuborish"}
</button>
```

Bu — real loyihalarda eng ko'p unutiladigan detal: `disabled={isSubmitting}` bo'lmasa, foydalanuvchi tugmani bir necha marta bosishi, va bir nechta bir xil buyurtma yuborilishi mumkin.

Keyingi bobda — statik ma'lumot (`data/rooms.ts`) o'rniga, tashqi manbadan (JSON fayl yoki API) ma'lumot olishni ko'ramiz.

9-BOB: useEffect va tashqi ma'lumot — statik JSON'dan API'gacha

Nega bu bob kerak

7-bobda `rooms` massivini to'g'ridan-to'g'ri kodga yozgan edik. Real loyihada bu ma'lumot ko'pincha o'zgarib turadi (narxlar, mavjudlik) — uni har safar kod yozib o'zgartirish noqulay. Yechim: ma'lumotni alohida JSON faylda saqlash yoki API'dan olish.

useEffect nima uchun kerak

Komponent birinchi marta ekranga chiqqanda biror amal bajarish kerak bo'lsa (masalan, ma'lumot yuklash), buning uchun `useEffect` ishlatiladi:

```
import { useState, useEffect } from "react";
import { Room } from "../data/rooms";

function RoomsSection() {
  const [rooms, setRooms] = useState<Room[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    fetch("/api/rooms")
      .then((res) => res.json())
      .then((data) => {
        setRooms(data);
        setIsLoading(false);
      })
      .catch(() => setIsLoading(false));
  }, []); // bo'sh massiv – faqat komponent birinchi marta chiqqanda ishga tushadi

  if (isLoading) return <p>Yuklanmoqda...</p>;

  return (
    <section className="rooms-section">
      {rooms.map((room) => (
        <RoomCard key={room.id} {...room} />
      ))}
    </section>
  );
}
```

useEffect 'ning ikkinchi argumenti — dependency array

`useEffect(() => {...}, [])` oxiridagi bo'sh massiv `[]` — bu "dependency array" deb ataladi va **qachon** effekt qayta ishga tushishini belgilaydi:

- `[]` (bo'sh) — faqat bir marta, komponent birinchi chiqqanda
- `[someValue]` — `someValue` o'zgarganda har safar
- massiv umuman bo'lmasa — har bir qayta chizishda (deyarli hech qachon kerak bo'lmaydi, va ko'pincha xatoga olib keladi)

Eng ko'p uchraydigan xato: dependency array'ni noto'g'ri to'ldirish, natijada effekt cheksiz tsiklda ishga tushadi (masalan, effekt ichida state'ni o'zgartirib, o'sha state dependency'da bo'lsa). Agar konsolda sayt "muzlab qolsa" yoki tarmoq so'rovlari cheksiz ketsa — birinchi tekshiradigan joy shu.

Agar hali backend/API yo'q bo'lsa

Ko'p hollarda, loyihaning boshida haqiqiy backend hali tayyor bo'lmaydi. Bunda `public/` papkasiga statik JSON fayl qo'yib, xuddi API'dan olayotgandek ishlatish mumkin:

```
public/data/rooms.json
```

```
useEffect(() => {
  fetch("/data/rooms.json")
    .then((res) => res.json())
    .then((data) => setRooms(data));
}, []);
```


Bu usul — kelajakda haqiqiy backend qo'shilganda, faqat `fetch` manzilini o'zgartirish kifoya, komponent kodi deyarli o'zgarishsiz qoladi.

Keyingi bobda — bir nechta sahifani (Bosh sahifa, Xonalar, Bog'lanish) qanday boshqarish, React Router yordamida.

10-BOB: Routing — bir nechta sahifa

Nega alohida kutubxona kerak

React o'zi "sahifalar" tushunchasini bilmaydi — u faqat komponentlarni chizadi. Ko'p sahifali sayt (URL o'zgarganda boshqa mazmun ko'rsatish) uchun **React Router** kutubxonasi ishlatiladi.

```
npm install react-router-dom
```

Asosiy sozlash

```
// main.tsx
import { BrowserRouter } from "react-router-dom";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

```
// App.tsx
import { Routes, Route } from "react-router-dom";
import Layout from "./components/Layout";
import HomePage from "./pages/HomePage";
import RoomsPage from "./pages/RoomsPage";
import ContactPage from "./pages/ContactPage";

function App() {
  return (
    <Layout>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/xonalar" element={<RoomsPage />} />
        <Route path="/boglanish" element={<ContactPage />} />
      </Routes>
    </Layout>
  );
}
```

<a> 'dan <Link> 'ga o'tish

4-bobda eslatilganidek, endi Header'dagi oddiy <a> teglarni <Link> komponentiga almashtiramiz:

```
import { Link } from "react-router-dom";

function Header() {
  return (
    <header>
      <nav>
        <Link to="/">Bosh sahifa</Link>
        <Link to="/xonalar">Xonalar</Link>
        <Link to="/boglanish">Bog'lanish</Link>
      </nav>
    </header>
  );
}
```

Farq: bosilganda butun sahifa qayta yuklanadi (barcha JS/CSS qaytadan yuklanadi, sekinroq). <Link to="/xonalar"> esa faqat kerakli qismni almashtiradi, sahifa "sakramaydi", tezroq va silliqroq tuyuladi.

Dinamik marshrutlar — bitta xonaning batafsil sahifasi

Agar har bir xona uchun alohida sahifa kerak bo'lsa (masalan /xonalar/1):

```
<Route path="/xonalar/:id" element={<RoomDetailPage />} />
```

```
// pages/RoomDetailPage.tsx
import { useParams } from "react-router-dom";
import { rooms } from "../data/rooms";

function RoomDetailPage() {
  const { id } = useParams();
  const room = rooms.find((r) => r.id === Number(id));

  if (!room) return <p>Xona topilmadi</p>;

  return (
    <div>
      <h1>{room.name}</h1>
      <p>{room.price.toLocaleString()} so'm / kecha</p>
    </div>
  );
}
```

Va `RoomCard` ichida `<Link to={/xonalar/${room.id} }>Batafsil</Link>` qo'shiladi.

Keyingi bobda — mobil qurilmalarga moslashuv, React komponentlarida responsive dizayn qanday tashkil qilinadi.

11-BOB: Responsive va mobil moslashuv

CSS yondashuvi o'zgarmaydi, tashkilot o'zgaradi

React interfeys mantig'ini boshqaradi, lekin responsive dizayn — bu hamon CSS vazifasi (media query, flexbox, grid). React bu yerda faqat komponentlarni qanday tashkil qilishga ta'sir qiladi.

CSS'ni komponentga bog'lash

Kichik-o'rta loyihalarda eng oddiy va tez yondashuv — har bir komponent uchun alohida CSS fayl:

```
components/  
├─ RoomCard.tsx  
└─ RoomCard.css
```

```
// RoomCard.tsx  
import "../RoomCard.css";  
  
function RoomCard({ name, price }: Room) {  
  return (  
    <div className="room-card">  
      <h3>{name}</h3>  
      <p>{price} so'm</p>  
    </div>  
  );  
}
```

```
/* RoomCard.css */  
.room-card {  
  display: flex;  
  flex-direction: column;  
  padding: 16px;  
}  
  
@media (min-width: 768px) {  
  .room-card {  
    flex-direction: row;  
  }  
}
```

Bu yondashuvning afzalligi: komponent va uning stili bir joyda, topish oson. Katta jamoaviy loyihalarda ko'pincha Tailwind CSS yoki CSS Modules kabi vositalar ishlatiladi, lekin bu qo'llanma doirasidan tashqarida — mustaqil o'rganish uchun mavzu sifatida qoldiramiz.

Grid layout — kartalar to'plami uchun

Xonalar, xizmatlar kabi kartalar to'plami uchun CSS Grid eng qulay:

```
.rooms-grid {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 20px;  
}  
  
@media (min-width: 640px) {  
  .rooms-grid {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
@media (min-width: 1024px) {  
  .rooms-grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

Bu — mobil qurilmada 1 ustun, planshetda 2, katta ekranda 3 ustunga avtomatik moslashadi.

Mobil-birinchi (mobile-first) yondashuv

Tavsiya etiladigan uslub — avval mobil versiyani yozish, keyin `min-width` media query orqali kattaroq ekranlar uchun qo'shimcha qoidalar qo'shish (yuqoridagi misoldagidek). Bu teskarisiga (`max-width` bilan desktop'dan boshlab pasaytirish) qaraganda kamroq CSS talab qiladi va telefon foydalanuvchilari (O'zbekistonda internet trafigining katta qismi) uchun tezroq yuklanadi.

Keyingi bobda — xatolar va bo'sh holatlarni (loading, error, hech narsa topilmadi) qanday chiroyli ko'rsatish.

12-BOB: Xatolar bilan ishlash va bo'sh holatlar

Nega bu ko'pincha unutiladi

Ko'p o'quv loyihalarida hamma narsa "ideal holatda" ishlaydi — internet doim bor, server doim javob beradi, ma'lumot doim to'liq. Real loyihada esa: internet uzilishi, server xatosi, yoki qidiruv natijasi bo'sh bo'lishi — kundalik holat. Bu holatlarni oldindan hisobga olmaslik — tugallanmagan loyihaning eng aniq belgisi.

Uch holatni har doim boshqaring

Har qanday tashqi ma'lumot bilan ishlaydigan komponentda kamida uchta holat bo'lishi kerak:

```
function RoomsSection() {
  const [rooms, setRooms] = useState<Room[]>([]);
  const [status, setStatus] = useState<"loading" | "success" | "error">("loading");

  useEffect(() => {
    fetch("/data/rooms.json")
      .then((res) => {
        if (!res.ok) throw new Error("Server xatosi");
        return res.json();
      })
      .then((data) => {
        setRooms(data);
        setStatus("success");
      })
      .catch(() => setStatus("error"));
  }, []);

  if (status === "loading") {
    return <p className="status-message">Yuklanmoqda...</p>;
  }

  if (status === "error") {
    return <p className="status-message error">Ma'lumotni yuklab bo'lmadi. Qayta urinib ko'ring.</p>;
  }

  if (rooms.length === 0) {
    return <p className="status-message">Hozircha xonalar mavjud emas.</p>;
  }

  return (
    <div className="rooms-grid">
      {rooms.map((room) => (
        <RoomCard key={room.id} {...room} />
      ))}
    </div>
  );
}
```

Diqqat qiling: `rooms.length === 0` alohida tekshirilgan — bu "xato" emas, balki "muvaffaqiyatli so'rov, lekin ma'lumot yo'q" holati. Ikkalasini bir xil ko'rsatish (masalan ikkalasida ham "Xatolik yuz berdi" deb yozish) foydalanuvchini chalg'itadi.

Forma xatolarini foydalanuvchiga tushunarli qilib ko'rsatish

8-bobda validatsiya xatolarini ko'rgan edik. Qo'shimcha qoida: xato xabari **nima noto'g'ri ekanini va uni qanday tuzatishni** aniq aytishi kerak. "Xato" yoki "Noto'g'ri" kabi umumiy xabarlar foydalanuvchini chalkashtiradi.

Yomon: `"Xato"` Yaxshi: `"Telefon raqam +998 bilan boshlanib, 9 ta raqamdan iborat bo'lishi kerak"`

Keyingi bobda — loyihani tayyor holga keltirib, internetga chiqaramiz: Vercel'ga deploy qilish va domen ulash.

13-BOB: Deploy qilish — Vercel'ga chiqarish

Nega Vercel

Vercel — React/Vite loyihalarini deploy qilish uchun eng keng tarqalgan bepul xizmatlardan biri. GitHub bilan bevosita integratsiyalashgan: kodni GitHub'ga yuklaganingizda, sayt avtomatik yangilanadi.

Qadamlar

1. Loyihani GitHub'ga yuklash:

```
git init
git add .
git commit -m "Birinchi versiya"
git branch -M main
git remote add origin https://github.com/foydalanuvchi-nomi/loyiha-nomi.git
git push -u origin main
```

2. Vercel'da loyihani ulash:

vercel.com saytida GitHub akkaunt orqali kiring, "New Project" tugmasini bosing, GitHub repozitoriyasini tanlang. Vercel avtomatik ravishda bu Vite loyihasi ekanini aniqlaydi va build sozlamalarini o'zi tanlaydi (`npm run build` , chiqish papkasi `dist`).

3. Deploy: "Deploy" tugmasini bosgach, bir necha daqiqada sayt `loyiha-nomi.vercel.app` manzilida jonli bo'ladi.

Domen ulash

Agar o'z domeningiz bo'lsa (masalan `mehmonuy.uz`):

1. Vercel loyihasida Settings → Domains bo'limiga o'ting
2. Domeningizni kiriting
3. Vercel sizga DNS yozuvlarini (odatda A yozuvi yoki CNAME) beradi
4. Bu yozuvlarni domen sotib olgan joyingizning (registrar) boshqaruv panelida qo'shing
5. DNS o'zgarishi tarqalishi uchun bir necha soatdan bir kungacha vaqt ketishi mumkin

Keyingi yangilanishlar qanday ishlaydi

Vercel ulanganidan keyin, jarayon juda oddiy bo'lib qoladi:

```
git add .
git commit -m "Xonalar bo'limi yangilandi"
git push
```

`git push` bajarilishi bilan, Vercel avtomatik ravishda yangi build qiladi va bir necha daqiqada o'zgarishlar jonli saytda ko'rinadi — qo'lda serverga fayl yuklash shart emas.

Muhit o'zgaruvchilari (agar API kalitlari bo'lsa)

Agar loyihangiz maxfiy kalitlar (masalan Telegram bot token) ishlatlsa, ularni hech qachon to'g'ridan-to'g'ri kodga yozmang. Buning o'rniga `.env` fayl va Vercel'ning "Environment Variables" bo'limidan foydalaning — bu mavzu murakkabroq bo'lgani uchun, agar loyihangizda backend/bot integratsiyasi bo'lsa, alohida o'rganishni tavsiya qilamiz.

Keyingi, so'nggi bobda — bu qo'llanmadan keyin nimani o'rganish kerakligi haqida yo'l xaritasi.

14-BOB: Keyingi qadamlar

Bu qo'llanma sizga bitta to'liq loyiha orqali React'ning asosiy qismlarini — komponentlar, props, state, ro'yxatlar, formalar, routing, deploy — amaliyotda ko'rsatdi. Lekin bu yakuniy nuqta emas, balki boshlang'ich.

Mustahkamlash uchun tavsiya etiladigan tartib:

- 1. Xuddi shu loyihani takrorlang, boshqa mavzu bilan.** Masalan mehmonxona o'rniga — kafe menyusi, portfolio sayt, kichik do'kon katalogi. Bu qo'llanmadagi bilimni "yodlash"dan "tushunish"ga aylantiradi.
 - 2. TypeScript'ni chuqurroq o'rganing** (agar hali sirtqi darajada bo'lsangiz). Katta loyihalarda TypeScript xatolarni yozish bosqichidayoq ushlab qoladi, bu ayniqsa ko'p komponentli loyihalarda vaqtni tejaydi.
 - 3. Global state boshqaruvi** — agar loyihangizda ko'p komponentlar orasida umumiy ma'lumot (masalan savat, foydalanuvchi holati) almashish kerak bo'lsa, React Context API (o'rnatilgan) yoki Zustand (yengil kutubxona) bilan tanishing.
 - 4. Next.js** — agar SEO muhim bo'lsa (qidiruv tizimlarida yuqori chiqish kerak) yoki server-side funkcionallik kerak bo'lsa, Next.js React'ning ustiga qurilgan freymvork sifatida keyingi mantiqiy qadam.
 - 5. Real loyihada xato qiling va tuzating.** Hech qanday qo'llanma sizni barcha xatolardan asray olmaydi — va bu yaxshi, chunki aynan xatoni topib tuzatish jarayoni eng ko'p o'rgatadi. Konsoldagi xato xabarini o'qishni, uni Google'da yoki AI yordamchi orqali qidirishni qo'rqmasdan qiling — bu ko'nikma tajribali dasturchilarning ham kundalik ishi.
-

ILOVA: Tez-tez uchraydigan xatolar

- "Cannot read properties of undefined"** Odatda ma'lumot hali yuklanmagan paytda uni o'qishga urinishda yuzaga keladi. Yechim: `isLoading` holatini tekshiring, yoki `room?.name` kabi optional chaining ishlatib.
- Komponent qayta-qayta chizilaveradi (infinite loop)** Odatda `useEffect` ichida dependency array noto'g'ri to'ldirilganda yuzaga keladi. Effekt ichida o'zgartirilayotgan state'ni dependency array'ga qo'shmang, agar bu cheksiz tsikl yaratsa.
- "Each child in a list should have a unique key prop" ogohlantirishi** `.map()` orqali chiqarilayotgan elementlarga `key` prop qo'shilmagan. 7-bobga qarang — `key` sifatida massiv indeksi emas, ma'lumotning o'ziga xos ID'sidan foydalaning.
- Forma maydoniga yozish ishlamayapti (input "muzlab qolgan")** `value` prop berilgan, lekin `onChange` orqali state yangilanmayapti — bu "controlled input"ning to'liqsiz yozilishi. 8-bobdagi to'liq misolga qarang.
- CSS o'zgarishlari ko'rinmayapti** Ko'pincha CSS fayl komponentga import qilinmagan, yoki `className` noto'g'ri yozilgan (CSS'da `.room-card`, lekin JSX'da `className="roomCard"`). Katta-kichik harflar va belgilarni solishtirib tekshiring.
- Deploy'dan keyin sahifa oq/bo'sh ko'rinadi** Odatda routing sozlamasida muammo (masalan Vercel'da SPA uchun rewrite qoidasi yo'q) yoki konsolda ko'rinadigan boshqa JavaScript xatosi. Brauzer konsolini (F12) tekshiring — deyarli har doim aniq xato xabari bo'ladi.

Ushbu qo'llanma "Sarfla" loyihasi doirasida tayyorlangan amaliy materialdir.